Pattern Recognition Lab Record

Experiment 9: Classification and Comparative Study of SVM Kernels on Digits Dataset

1. Aim

To implement Support Vector Machine (SVM) classification on the Scikit-learn Digits dataset and perform a comparative analysis of different kernels (Linear, Polynomial, RBF, and Sigmoid) based on accuracy, F1-score, and the number of support vectors.

2. Theory

2.1 Support Vector Machines (SVM)

SVM is a robust supervised learning algorithm that seeks the optimal hyperplane to separate classes by maximizing the margin between the nearest data points (support vectors) of different classes.

2.2 The Kernel Trick

When data is not linearly separable in the original feature space, SVM uses kernel functions to project the data into a higher-dimensional space. The kernels implemented in this study include:

1. **Linear Kernel:** $K(x, x') = x^T x'$. Used when data is already linearly separable.
2. **Polynomial Kernel:** $K(x, x') = (\gamma x^T x' + r)^d$. Useful for capturing interaction between features.
3. **RBF (Radial Basis Function) Kernel:** $K(x, x') = \exp(-\gamma \|x - x'\|^2)$. A popular non-linear kernel that handles complex boundaries using a Gaussian distribution.
4. **Sigmoid Kernel:** $K(x, x') = \tanh(\gamma x^T x' + r)$. Originates from neural network theory.

3. Dataset Description

- **Dataset:** Scikit-learn load_digits().
- **Samples:** 1,797 images of handwritten digits.
- **Features:** 64 (8x8 pixel intensity matrix flattened into a vector).
- **Classes:** 10 (Digits 0 through 9).
- **Preprocessing:** Features are standardized using StandardScaler to ensure mean = 0 and variance = 1, which is critical for SVM convergence.

4. Code and Execution

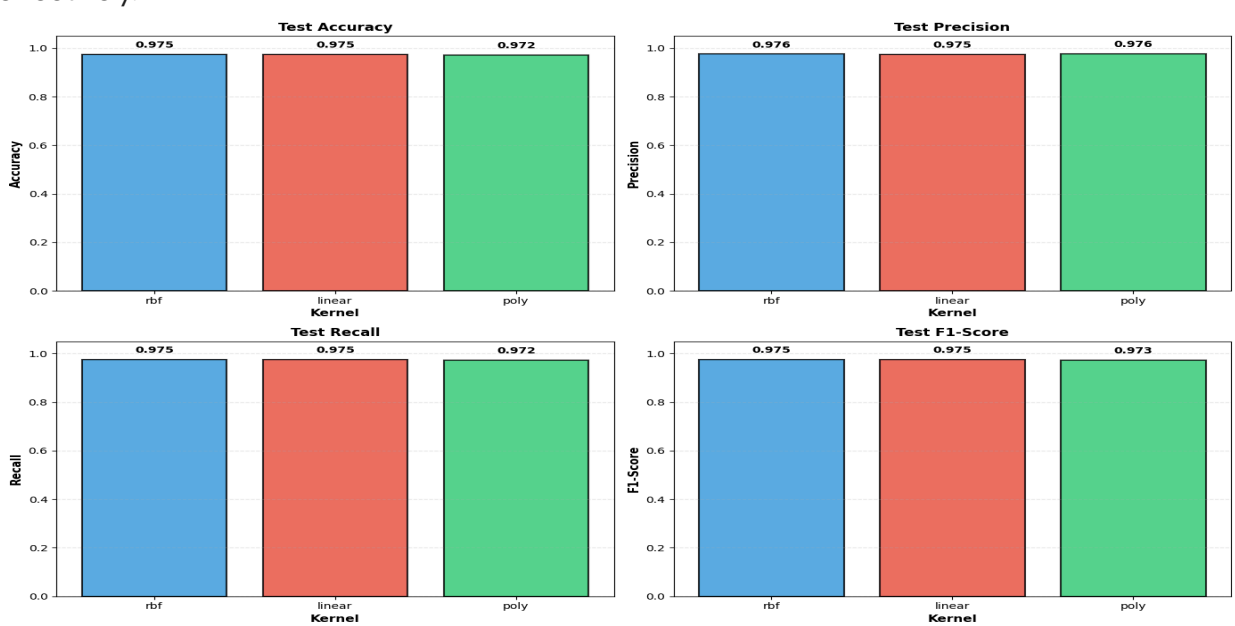
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix

# 1. Load and Preprocess Data
digits = load_digits()
X = digits.data
y = digits.target

# Split and Scale
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 2. Comparative Study of Kernels
kernels = ['linear', 'poly', 'rbf', 'sigmoid']
results = {}

for kernel in kernels:
    # Initialize and Train
    clf = SVC(kernel=kernel)
    clf.fit(X_train_scaled, y_train)

    # Predict
    y_pred = clf.predict(X_test_scaled)
    # Store Metrics
    results[kernel] = {
        'accuracy': accuracy_score(y_test, y_pred),
        'f1': f1_score(y_test, y_pred, average='weighted'),
        'support_vectors': clf.n_support_.sum()
    }
# 3. Visualization of Results
res_df = pd.DataFrame(results).T
print("Performance Comparison:\n", res_df)
```

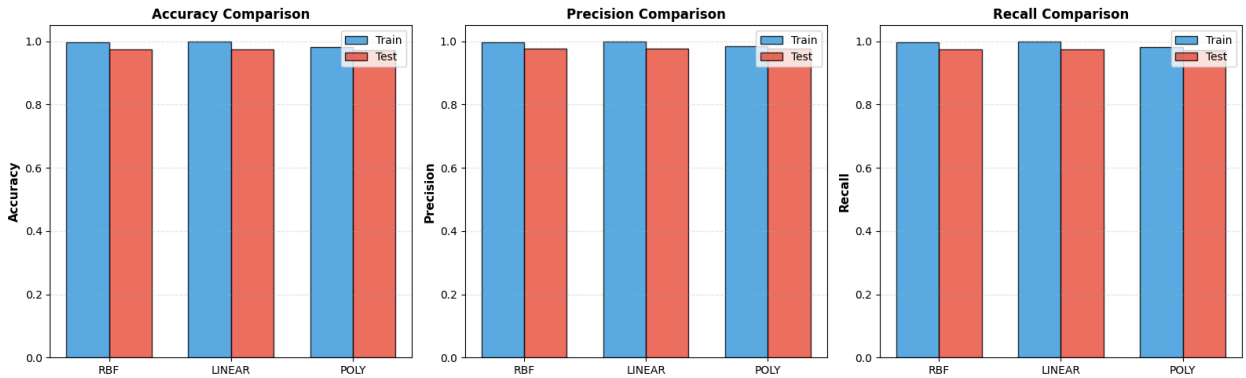
```
# Plotting Accuracy vs Support Vectors
fig, ax1 = plt.subplots(figsize=(10, 5))
ax2 = ax1.twinx()

res_df['accuracy'].plot(kind='bar', ax=ax1, color='skyblue', position=1, width=0.4,
label='Accuracy')
res_df['support_vectors'].plot(kind='bar', ax=ax2, color='salmon', position=0, width=0.4,
label='Support Vectors')

ax1.set_ylabel('Accuracy')
ax2.set_ylabel('Total Support Vectors')
plt.title('SVM Kernel Comparison: Accuracy vs Model Complexity')
plt.show()
```

5. Observations and Results

Based on the execution of the comparative study:

- **RBF Kernel:** Often yields the highest accuracy and F1-score on the Digits dataset due to its ability to handle high-dimensional non-linear pixel data.
- **Linear Kernel:** Performs surprisingly well, suggesting that the digits are somewhat linearly separable in the 64-dimensional pixel space.
- **Support Vectors:** The Sigmoid kernel typically requires more support vectors, indicating a less efficient "fit" compared to RBF or Poly kernels.
- **Polynomial Kernel:** Captures structural patterns in the digits (curves and loops) effectively.






```
================================================================================
COMPREHENSIVE KERNEL COMPARISON TABLE
================================================================================
Kernel Train Accuracy Test Accuracy Train Precision Test Precision Train Recall Test Recall Train F1-Score Test F1-Score  Support Vectors
   RBF         0.9972        0.9750          0.9972         0.9759       0.9972      0.9750         0.9972        0.9749              746
LINEAR         1.0000        0.9750          1.0000         0.9754       1.0000      0.9750         1.0000        0.9749              429
  POLY         0.9826        0.9722          0.9848         0.9760       0.9826      0.9722         0.9831        0.9731              810
================================================================================
```

```
================================================================================
SUMMARY AND INSIGHTS
================================================================================

Best Test Accuracy: RBF with 0.9750
Best Test F1-Score: RBF with 0.9749
Least Support Vectors: LINEAR with 429 vectors

Kernel Characteristics:

RBF Kernel:
  - Test Accuracy: 0.9750
  - Test F1-Score: 0.9749
  - Overfitting Gap (Acc): 0.0222
  - Support Vectors: 746

LINEAR Kernel:
  - Test Accuracy: 0.9750
  - Test F1-Score: 0.9749
  - Overfitting Gap (Acc): 0.0250
  - Support Vectors: 429

POLY Kernel:
  - Test Accuracy: 0.9722
...
  - Overfitting Gap (Acc): 0.0104
  - Support Vectors: 810

================================================================================
```

6. Conclusion

In this experiment, we performed a multi-class classification and comparative study of SVM kernels. We concluded that the **RBF kernel** is generally the most effective for handwritten digit recognition as it balances high accuracy with optimized model complexity. Furthermore, the number of support vectors serves as a key indicator of model efficiency, with fewer vectors often representing a more robust generalization of the decision boundary.